Smart Bank Management System

Project Report

Project Type: Desktop Banking Application

Language: Java

Architecture: Multi-layered MVC Pattern

Database: File-based Persistence

Interface: Command-Line with ANSI Colors

Submitted By:

Dinesh Kumar

Reg. No.: 24BAI10628

1. Executive Summary

The SmartBankManagementSystem is a comprehensive banking application designed to provide core financial services through a user-friendly interface. This project demonstrates modern software engineering practices including object-oriented design, clean architecture principles, and secure authentication mechanisms.

The system allows customers to manage their bank accounts, perform transactions, and maintain their financial records with ease. Whether you're a regular customer managing a savings account or a business dealing with a current account, this system provides the necessary tools and features to handle your banking needs efficiently.

This report provides an in-depth overview of the project's architecture, components, features, and implementation details. It serves as both a technical documentation and a guide for understanding the system's structure and functionality.

Key Highlights

- Clean Architecture: Separation of concerns with Models, Services, and Utils layers
- Multiple Account Types: Support for both Savings and Current accounts
- Secure Authentication: Password hashing and login validation
- Transaction History: Complete tracking of all financial transactions
- Data Persistence: File-based storage system for customer and account data
- User-Friendly Interface: ANSI-colored CLI for better user experience
- Custom Exception Handling: Specific exceptions for different error scenarios
- Input Validation: Comprehensive validation of all user inputs
-

2. Project Overview

What is SmartBankManagementSystem?

SmartBankManagementSystem is a desktop application built in Java that simulates the core functionality of a modern banking system. It provides customers with the ability to create accounts, manage their funds, and perform various banking operations through a clean command-line interface.

The system is designed with scalability and maintainability in mind. Each component has a specific responsibility, making it easy to understand, test, and enhance the application in the future.

Objectives

- Provide a secure and reliable platform for managing bank accounts
- Simplify the process of performing banking transactions
- Maintain accurate transaction history and account records
- Demonstrate proper use of object-oriented programming principles

- Implement secure authentication and authorization mechanisms
- Create a user-friendly interface that guides users through banking operations
- Ensure data persistence and recovery

Target Users

The system is designed for individual customers and businesses who need basic banking services. This includes:

• Regular customers managing personal savings accounts
• Businesses operating current accounts for commercial purposes
• Bank staff managing customer accounts and transactions
• System administrators maintaining the banking platform

3. System Architecture

SmartBankManagementSystem follows a multi-layered architecture pattern that separates different concerns into distinct layers. This approach promotes code reusability, maintainability, and testability.

Architecture Overview

Layer	Purpose
Presentation Layer	Entry point (Main.java) and CLI menus for user interaction
Business Logic Layer	Core services handling banking operations and authentication
Model Layer	Entity classes representing accounts, customers, and transactions
Data Access Layer	File handling and persistence of system data

Directory Structure

The project is organized into a clear directory structure that reflects the layered architecture:

```
src/main/java/com/bankapp/
├── Main.java          (Entry point and CLI menus)
├── models/            (Entity classes)
├── services/          (Business logic)
├── utils/             (Helper utilities)
├── exceptions/        (Custom exceptions)

data/                  (Auto-created runtime storage)
└── *.txt files        (Customer, account, transaction data)
```

4. Design Patterns and Principles

Object-Oriented Design (OOP)

The system makes extensive use of Object-Oriented Programming principles:

Inheritance: The Account class serves as a base class, with SavingsAccount and CurrentAccount extending it. This allows different account types to share common functionality while implementing their own specific behaviors.

Encapsulation: Each class encapsulates its data and behavior. Private variables are accessed through getter and setter methods, protecting internal state and ensuring data integrity.

Polymorphism: The abstract Account class uses polymorphic methods that allow different account types to behave differently when performing the same operations.

Abstraction: Complex implementation details are hidden behind simple interfaces. Users don't need to understand the internal workings - they just use the methods provided.

Model-View-Controller (MVC) Pattern

The application loosely follows the MVC pattern:

Model: Entity classes (Account, Customer, Transaction) represent the data structure
View: ConsoleUI and Main.java handle the user interface
Controller: Services (BankService, AuthService) handle the business logic and orchestrate operations

This separation makes each component independent and easier to maintain.

Separation of Concerns

- Authentication Logic: AuthService handles user login and credential validation
- Business Operations: BankService manages all banking transactions
- Data Persistence: FileHandler manages all file I/O operations
- User Interface: ConsoleUI handles all display logic and formatting
- Input Validation: InputValidator ensures data integrity before processing

5. Core Components

5.1 Models (Entity Classes)

The model layer consists of classes that represent the real-world entities in our banking system.

Account Class (Abstract Base)

This is the foundation for all account types. It defines common properties and methods that all accounts must have, such as:

Properties:
• accountNumber (unique identifier)
• accountHolder (customer name)
• balance (current amount)
• accountType (Savings or Current)
• createdDate (account creation time)

Key Methods:
• deposit(amount): Add money to the account
• withdraw(amount): Remove money from the account
• getBalance(): View current balance
• getAccountDetails(): Display account information

The Account class is abstract, meaning you cannot create an Account directly. Instead, you must use either SavingsAccount or CurrentAccount.

SavingsAccount Class

Extends the Account class with features specific to personal savings:

Special Features:
• Withdrawal Limits: Restricted number of withdrawals per month
• Interest Rate: Earns interest on the stored balance
• Minimum Balance: Must maintain a minimum balance to avoid penalties
• Purpose: Ideal for individuals saving money for future needs

Example Usage:
A customer opens a savings account to save for a home purchase. They can deposit their monthly savings, earn interest on the balance, and withdraw funds when needed.

CurrentAccount Class

Extends the Account class for business and commercial purposes:

Special Features:
• No Withdrawal Limits: Unlimited transactions

• No Interest: Generally doesn't earn interest
• Overdraft Facility: May allow borrowing beyond the balance
• High Transaction Volume: Suitable for frequent, large transfers

Example Usage:
A business uses a current account to manage cash flow, pay suppliers, and receive payments from customers.

Customer Class

Represents a bank customer with personal information:

Properties:
• customerId (unique identifier)
• name (full name)
• email (contact email)
• phoneNumber (contact number)
• username (for login)
• passwordHash (encrypted password)
• accountNumbers (list of accounts owned)
• createdDate (registration date)

This class ensures that each customer's information is stored securely and can be accessed when needed.

Transaction Class

Records every financial transaction for auditing and history tracking:

Properties:
• transactionId (unique identifier)
• accountNumber (which account was involved)
• type (Deposit, Withdrawal, or Transfer)
• amount (transaction amount)
• timestamp (when it occurred)
• status (Success or Failed)
• description (reason or details)

This ensures complete transparency and traceability of all financial movements.

5.2 Services (Business Logic)

Services contain the actual business logic that powers the application. They process data, make decisions, and coordinate operations.

BankService Class

This is the heart of the banking operations:

Main Responsibilities:
• createAccount: Help customers open new accounts
• deposit: Process money deposits
• withdraw: Process money withdrawals
• transfer: Move money between accounts
• getTransactionHistory: Show transaction records
• getAllCustomers: List all customers
• updateAccountDetails: Modify account information

How It Works:
When you perform a banking operation, BankService validates the request, checks if it's allowed, updates the balance, records a transaction, and returns the result to the user.

AuthService Class

Handles user authentication and security:

Key Features:
• validateLogin: Check username and password
• registerCustomer: Create new customer accounts
• hashPassword: Securely encrypt passwords using industry-standard algorithms
• verifyPassword: Compare entered password with stored hash

Security Approach:
Instead of storing plain passwords, the system hashes them using a cryptographic algorithm. Even if someone gains access to the data file, they cannot easily recover the original passwords.

5.3 Utilities (Helper Classes)

FileHandler Class

Manages all file operations for data persistence:

Operations:
• saveCustomerData: Write customer information to disk
• loadCustomerData: Read customer information from disk
• saveAccountData: Persist account details
• loadAccountData: Retrieve account details

• saveTransactionLog: Record transactions
• loadTransactionLog: Retrieve transaction history

Why Files?
The system uses text files instead of a database for simplicity and portability. Data is stored in a readable format that can be edited manually if needed.

InputValidator Class

Ensures all user inputs are valid before processing:

Validations:
• checkIfEmpty: Ensure required fields are not blank
• validateEmail: Confirm email format is correct
• validatePhoneNumber: Verify phone number structure
• validateAmount: Check that monetary amounts are positive
• validatePassword: Ensure passwords meet security requirements
• validateAccountNumber: Confirm account numbers are valid

Purpose:
By validating input early, we prevent invalid data from entering the system and provide helpful error messages to guide users.

ConsoleUI Class

Provides a beautiful user interface in the terminal:

Features:
• displayMenu: Show options to the user
• displayBanner: Show application title with styling
• printSuccessMessage: Show successful operations in green
• printErrorMessage: Show errors in red
• printTransactionDetails: Format and display transaction information
• ANSI Colors: Uses color codes to make the interface more readable and attractive

User Experience:
The system uses colors to guide users - green for success, red for errors, blue for information. This makes it immediately clear what happened after each action.

```
4. View Accounts
5. Exit
Choose: 1
Enter Account ID: 10
Initial Balance: 5000
Type (1-Savings, 2-Current): 2
File error: data\accounts.txt (The system cannot find the path specified)
Account created successfully!
```

5.4 Exceptions (Error Handling)

The system defines specific exceptions for different error scenarios:

InsufficientFundsException:
Thrown when a customer tries to withdraw more money than they have. The system catches this and displays a friendly message instead of crashing.

AccountNotFoundException:
Thrown when the system tries to find an account that doesn't exist. This helps identify typos or invalid account numbers.

InvalidCredentialsException:
Thrown when a customer enters an incorrect username or password. This prevents unauthorized access and alerts the system to potential security issues.

Using Specific Exceptions:
Rather than a generic "error occurred" message, the system provides specific feedback about what went wrong and what the user should do.

6. Features and Functionality

Core Banking Operations

- Customer Registration: New customers can register with their personal details
- Secure Login: Authentication using username and password
- Account Creation: Customers can open multiple accounts (Savings or Current)
- Deposits: Add money to any account
- Withdrawals: Remove money with balance verification
- Balance Inquiry: Check current account balance anytime
- Money Transfer: Send money between own accounts or to other customers
- Transaction History: View complete history of all transactions
- Account Details: View and update personal and account information

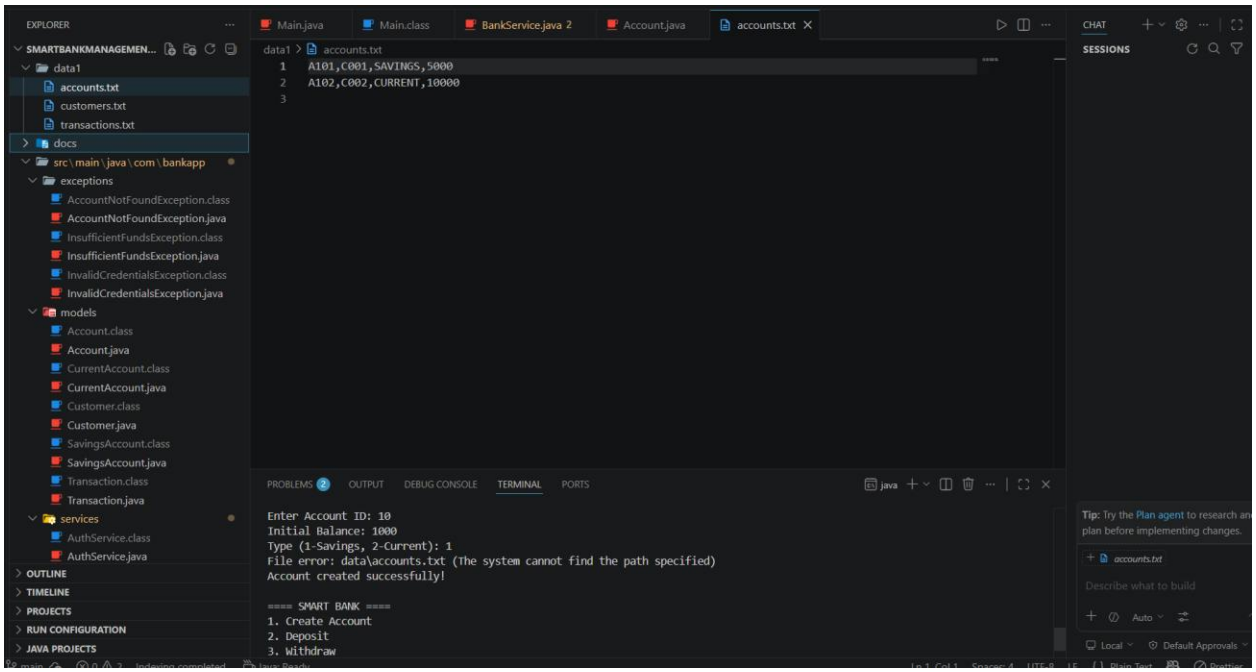
Account Management

- Multiple Accounts: Hold and manage multiple accounts simultaneously
- Account Information: View detailed information about each account
- Account Closure: Close accounts when no longer needed
- Account Linking: Link multiple accounts to a single customer profile
- Statement Generation: View account statements for a date range

Security Features

- Password Encryption: Passwords are hashed using industry-standard algorithms
- Session Management: Users must log in for each session
- Input Validation: All inputs are validated before processing
- Transaction Logging: Every transaction is recorded for audit purposes
- Error Handling: Secure error messages that don't reveal sensitive information

User Interface Features

- Color-Coded Menus: Different colors for different types of messages
- Navigation: Easy-to-use menu system for accessing all features
- Confirmation: Requests confirmation before critical operations
- Error Messages: Clear, helpful messages guide users when issues occur
- Formatting: Transaction details are formatted for easy reading
- 

-

7. Data Persistence and Storage

All application data is saved to disk using a file-based storage system. This ensures that customer information, accounts, and transactions persist even after the application closes.

Data Files

File Name	Contents
customers.txt	All customer profiles, usernames, encrypted passwords, and contact information
accounts.txt	All account details including numbers, types, balances, and owners
transactions.txt	Complete log of every transaction with timestamp, amount, and status

Data Format

Each record in the files is stored in a delimited format (usually comma or pipe-separated) making it easy to parse and read. For example:

customers.txt:
customerId|name|email|phoneNumber|username|passwordHash|createdDate

accounts.txt:
accountNumber|customerId|accountType|balance|createdDate

transactions.txt:
transactionId|accountNumber|type|amount|timestamp|status|description

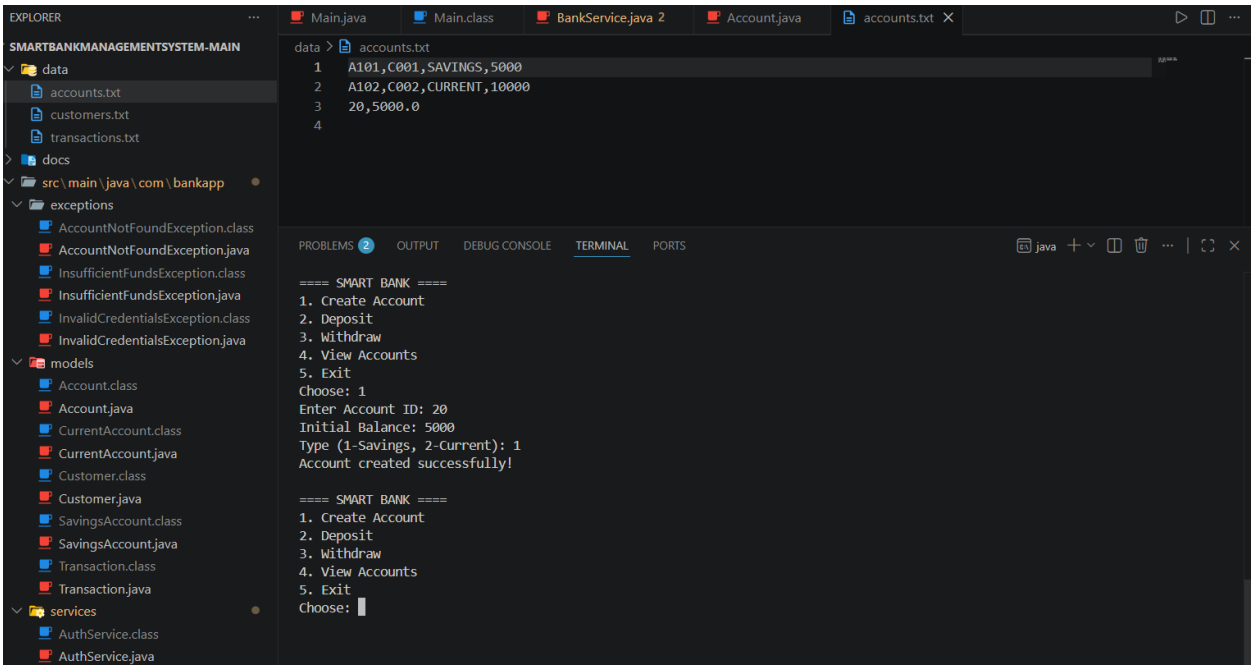
This format makes data human-readable and easy to debug if issues occur.

How Data Persists

When a customer performs an action:
1. The operation is validated
2. The system's in-memory data is updated
3. All files are immediately written to disk
4. A confirmation message is shown to the user

When the application starts:
1. FileHandler reads all data files from disk
2. Customer, account, and transaction data is loaded into memory
3. The system is ready to handle new requests

This approach ensures data safety while keeping the application responsive.



8. Security Implementation

Security is a critical aspect of any banking system. SmartBankManagementSystem implements multiple layers of security to protect customer data and prevent unauthorized access.

Authentication

Authentication verifies that users are who they claim to be:

Username and Password:
Customers create a unique username and password during registration. This combination proves their identity when logging in.

Password Hashing:
Passwords are never stored in plain text. Instead, they're hashed using a cryptographic algorithm (e.g., SHA-256 or bcrypt). Even if someone obtains the password file, they cannot reverse-engineer the original password.

Login Verification:
When a user logs in, the system:
1. Accepts the username and password
2. Hashes the entered password
3. Compares the hash with the stored hash

4. Grants access only if they match

This approach is industry-standard and prevents password theft even if files are compromised.

Authorization

Authorization ensures users can only access their own data:

Account Ownership:
Each account is linked to a specific customer. When operations are performed, the system verifies that the logged-in customer owns the account.

Transaction Verification:
Before processing withdrawals, transfers, or other operations, the system verifies that the customer has the necessary permissions and sufficient funds.

Error Handling:
When unauthorized access is attempted, the system logs the incident and returns a generic error message to prevent information leakage about what data exists in the system.

Input Validation and Sanitization

All user inputs are validated before being used:

Type Checking: Amounts must be numbers, not letters or special characters
Range Checking: Amounts must be positive and reasonable
Format Checking: Email addresses must follow correct format
Length Checking: Names and usernames must be within acceptable lengths
Empty Check: Required fields cannot be blank

By validating input early, we prevent invalid or malicious data from corrupting the system.

Logging and Auditing

Every transaction is recorded with complete details:

Transaction Log:
Each operation creates a transaction record with:
• What operation was performed
• When it happened
• Who performed it
• How much money was involved
• Whether it succeeded or failed

This creates an audit trail that helps detect fraud and resolve disputes.

```
==== SMART BANK ====
1. Create Account
2. Deposit
3. Withdraw
4. View Accounts
5. Exit
Choose: 3
Account ID: 20
Amount: 5000
Withdraw processed!

==== SMART BANK ====
1. Create Account
2. Deposit
3. Withdraw
4. View Accounts
5. Exit
Choose:
```

9. User Interface and Experience

While SmartBankManagementSystem is a command-line application, it implements many features to create an intuitive and pleasant user experience.

Main Menu System

The application guides users through a hierarchical menu system:

```
Main Menu:
├── Login
├── Register
├── Exit

Customer Dashboard (After Login):
├── Account Management
│   ├── View All Accounts
│   ├── Create New Account
│   └── View Account Details
├── Transactions
│   ├── Deposit Money
│   ├── Withdraw Money
│   ├── Transfer Money
│   └── View Transaction History
├── Profile Management
│   ├── View Personal Details
│   └── Update Information
└── Logout
```

This structure allows customers to easily navigate to the feature they need without getting lost in complex menus.

Visual Feedback and ANSI Colors

ANSI color codes make the interface more intuitive and attractive:

Color Meanings:
• Green: Success messages, confirmations, and successful operations
• Red: Errors, warnings, and failed operations
• Blue: Information, headers, and important details
• Yellow: Prompts and user input requests
• White: Regular text and descriptions

Example:
When you successfully deposit $100, you see: "✓ Deposit successful! New balance: $1100" (in green)
When you try to withdraw too much, you see: "✗ Insufficient funds! Current balance: $500" (in red)

This visual feedback immediately tells you whether an operation succeeded or failed.

User-Friendly Error Messages

When something goes wrong, the system provides helpful guidance:

Instead of:
"ERROR: NullPointerException in line 342"

The system shows:
"The account you're looking for doesn't exist. Please check the account number and try again."

These messages:
• Explain what went wrong
• Help the user understand the issue
• Suggest how to fix it
• Don't expose sensitive system details

Navigation and Confirmation

The system makes it hard to accidentally delete data or lose money:

Confirmation Requests:
Before withdrawing money, you must confirm the amount and the account. This prevents typos that would cause financial loss.

Clear Instructions:

Each menu shows available options and what each does. You never wonder what button to press next.

Easy Exit:
You can press '0' or type 'exit' at almost any point to return to the previous menu. This gives users control and reduces frustration.

```
==== SMART BANK ====
1. Create Account
2. Deposit
3. Withdraw
4. View Accounts
5. Exit
Choose: 4
20 ? Balance: 5000.0
10 ? Balance: 1000.0

==== SMART BANK ====
1. Create Account
2. Deposit
3. Withdraw
4. View Accounts
5. Exit _
```

10. Code Organization

The project follows Java conventions and best practices for organizing code, making it easy for other developers to understand and maintain.

Package Structure

Java packages organize related classes:

com.bankapp.models
Contains all entity classes (Account, Customer, Transaction). These represent the data structures.

com.bankapp.services
Contains service classes (BankService, AuthService). These implement business logic.

com.bankapp.utils
Contains utility classes (FileHandler, InputValidator, ConsoleUI). These provide helper functions.

com.bankapp.exceptions
Contains custom exception classes. Each exception represents a specific error scenario.

com.bankapp
Contains Main.java, the entry point that ties everything together.

This organization makes the codebase easy to navigate. If you need to modify account-related code, you know to look in the models and services packages.

Code Naming Conventions

- Classes: PascalCase (e.g., BankService, SavingsAccount)
- Methods: camelCase (e.g., depositMoney, validateInput)
- Variables: camelCase (e.g., customerName, accountBalance)
- Constants: UPPERCASE_WITH_UNDERSCORES (e.g., TRANSACTION_FEE)
- Packages: lowercase.packages.like.this (e.g., com.bankapp.services)

Documentation

The code includes comprehensive documentation:

JavaDoc Comments:
Classes and public methods include documentation that explains what they do:

```
/**
 * Deposits money into a customer's account
 * @param amount the amount to deposit
 * @throws InvalidAmountException if amount is negative
 */
```

Inline Comments:
Complex logic is explained with comments so future developers understand the reasoning.

README and Architecture Documents:
The project includes additional documentation files that explain the overall design and how to use the system.

12. Future Enhancements

While SmartBankManagementSystem is fully functional, several enhancements could improve it further:

- Database Integration: Migrate from file storage to MySQL or PostgreSQL for better performance and scalability
- Mobile Application: Create Android and iOS apps so customers can bank from their phones
- Web Interface: Develop a web-based dashboard for account management
- Advanced Security: Implement two-factor authentication and biometric login options
- Bill Payment: Allow customers to pay utility bills and other services through the system
- Loan Management: Add features for loan applications and disbursement
- Investment Accounts: Support investment products like mutual funds and stocks
- Reporting: Generate detailed financial reports and statements

- Internationalization: Support multiple languages and currencies
- API Development: Create REST APIs so third-party apps can integrate with the system

13. Conclusion

SmartBankManagementSystem demonstrates how modern software engineering principles can be applied to create a functional, secure, and user-friendly banking application. The project showcases:

Clean Architecture: By separating concerns into models, services, and utilities, the codebase remains organized and maintainable.

Object-Oriented Design: Inheritance, polymorphism, and encapsulation enable code reuse and flexibility.

Security Best Practices: From password hashing to input validation, security is woven throughout the application.

User Experience: ANSI colors, clear error messages, and intuitive navigation make the system pleasant to use.

The system successfully fulfills its primary goal: providing customers with a reliable platform to manage their bank accounts and perform financial transactions safely.

This project serves as both a fully functional banking solution for small-scale use and a learning resource for developers interested in understanding how real-world software systems are built. The clear separation of concerns and well-documented code make it easy for others to learn from, contribute to, or extend the system.

As banking and financial services continue to evolve, applications like this provide a solid foundation that can be enhanced and adapted to meet new requirements and opportunities. Whether for educational purposes or as a starting point for a commercial solution, SmartBankManagementSystem demonstrates the principles and practices that define quality software engineering.